



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor Jodie Soesila
Minggu ke / Materi	07 / Percabangan dan Perulangan Kompleks

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

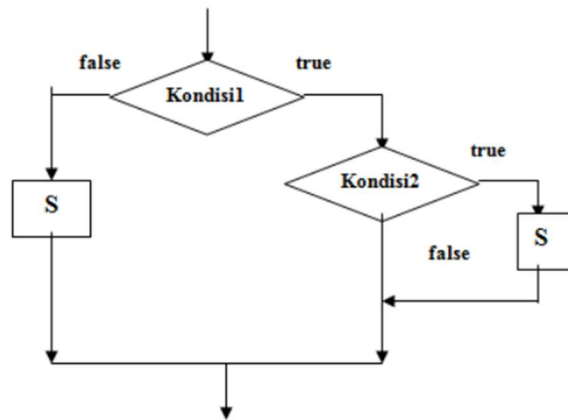
BAGIAN 1: MATERI MINGGU INI (40%)

Struktur Percabangan Kompleks

Percabangan kompleks adalah percabangan dengan banyak kondisi dan alternatif, di mana setiap kondisi dapat memiliki lebih dari satu perintah.

```
if kondisi1:  
    if kondisi2:  
        S  
        S  
else:  
    S  
    S
```

Gambar flowchartnya:

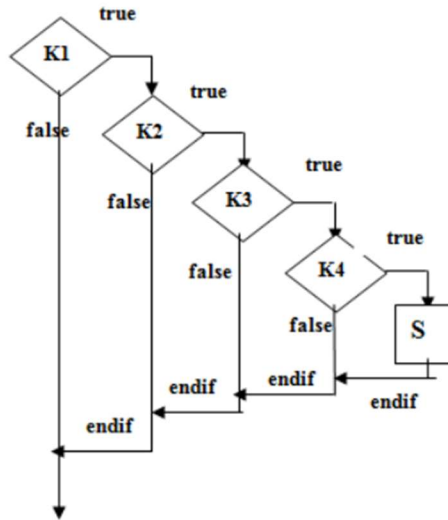


Gambar 6.1 Flowchart Percabangan Kompleks Bentuk 1. Sumber: <https://sl1nk.com/aiVqB>

Percabangan kompleks bentuk 2:

```
if kondisi1:  
    if kondisi2:  
        if kondisi3:  
            if kondisi4:  
                S
```

Gambar flowchart percabangan kompleks bentuk 2:



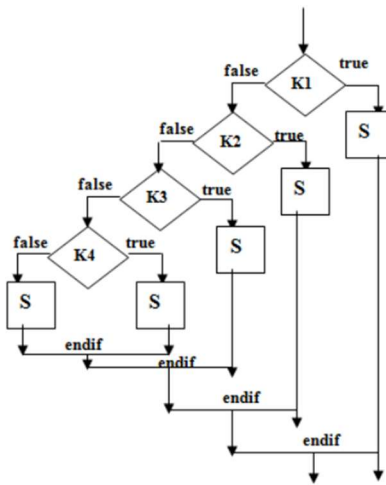
Gambar 6.2 Flowchart Percabangan Kompleks Bentuk 2. Sumber: <https://sl1nk.com/aiVqB>

Percabangan kompleks bentuk 3:

```

if kondisi1:
    S
else:
    if kondisi2:
        S
    else:
        if kondisi3:
            S
        else:
            if kondisi4:
                S
            else:
                S
  
```

Gambar flowchart percabangan kompleks bentuk 3:



Gambar 6.3: Flowchart Percabangan Kompleks Bentuk 3. Sumber: <https://sl1nk.com/aiVqB>

Struktur Perulangan Kompleks Break

Perintah ini untuk menghentikan proses perulangan yang terjadi. Biasanya disebabkan oleh suatu kondisi yang menggunakan syntax `if`.

```

for i in range(1000):
    print(i)
    if i == 10:
        break
  
```

Outputnya:

```

0
1
2
3
4
5
6
7
8
9
  
```

10

Continue

Proses perulangan kembali ke awal mula, dengan mengabaikan statement-statement berikutnya.

Contoh program continue :

```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```

Outputnya:

```
0  
1  
2  
3  
4  
6  
7  
8  
9
```

Perulangan Bertingkat yaitu perulangan yang di dalamnya terdapat perulangan lain sehingga prosesnya menjadi lebih lama. Struktur ini biasanya digunakan pada algoritma tertentu, seperti pengolahan matriks (array 2 dimensi), game board seperti catur dan minesweeper, serta pengolahan citra digital. Secara umum, perulangan kompleks digunakan untuk menyelesaikan masalah yang memiliki pola grid atau berbentuk kotak dengan panjang dan lebar tertentu.

:

Program 1:

```
for i in range(m):  
    <lakukan perintah ini>  
    <lakukan perintah itu>
```

Program 2:

```
for j in range(n):  
    <lakukan perintah ini>  
    <lakukan perintah itu>
```

Jika perulangan pada program 2 "dimasukkan" ke dalam perulangan program 1, maka menjadi:

```
for i in range(m):  
    for j in range(n):  
        <lakukan perintah ini di inner>  
        <lakukan perintah itu di inner>  
    <lakukan perintah lain di outer>  
    <lakukan perintah lain lagi di outer>
```

Perulangan **for i** berperan sebagai *outer loop*, sedangkan **for j** sebagai *inner loop*. Alur kerjanya adalah setiap kali *outer loop* dijalankan dari 1 hingga m, maka *inner loop* akan dieksekusi sebanyak n kali pada setiap iterasinya. Dengan demikian, jumlah total eksekusi *inner loop* bergantung pada banyaknya pengulangan pada *outer loop*. Perhatikan contoh berikut:

```
while(i<=m):  
    while(j<=n):  
        <lakukan perintah ini di inner>  
        <lakukan perintah itu di inner>  
    <lakukan perintah lain di outer>  
    <lakukan perintah lain lagi di outer>
```

Pada struktur perulangan ini, bagian **while i** berperan sebagai *outer loop* (perulangan luar), sedangkan **while j** berfungsi sebagai *inner loop* (perulangan dalam). Mekanisme kerjanya adalah setiap kali *outer loop* dijalankan, mulai dari 1 hingga m, maka di dalam setiap iterasi tersebut *inner loop* akan dieksekusi sebanyak n kali. Dengan demikian, *inner loop* akan terus berulang mengikuti jumlah perulangan yang terjadi pada *outer loop*.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github: https://github.com/chinorjodie/71251187_chinor.git

SOAL 1

Source code:

```
n = int(input("Masukkan bilangan: "))

def bilangan_prima(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def cari_woy(n):
    for i in range (n - 1, 1, -1):
        if bilangan_prima(i):
            return i
    return None

hasil = cari_woy(n)

if hasil:
    print(f"maka prima terdekat < {n} adalah {hasil} ")
else:
    print("Tidak ada bilangan prima di bawah", n)
```

Output:

```
● PS D:\71251187_chinor\minggu07> py .\soal01.py
input n = 12
maka prima terdekat < 12 adalah 11
● PS D:\71251187_chinor\minggu07> py .\soal01.py
input n = 21
maka prima terdekat < 21 adalah 19
○ PS D:\71251187_chinor\minggu07> █
```

Penjelasan:

Pertama, pengguna memasukkan nilai **n**, lalu fungsi **bilangan_prima(n)** digunakan untuk mengecek apakah suatu angka merupakan bilangan prima dengan cara menguji apakah angka tersebut memiliki faktor selain 1 dan dirinya sendiri (pengecekan sampai kuadratnya saja supaya efisien). Selanjutnya, fungsi **cari_woy(n)** melakukan perulangan mundur dari n-1 hingga 2 untuk menemukan angka pertama yang memenuhi kondisi bilangan prima menggunakan fungsi sebelumnya. Begitu ditemukan, nilai tersebut langsung dikembalikan. Hasil pencarian disimpan dalam variabel **hasil**, kemudian program akan menampilkan pesan bahwa tidak ada bilangan prima di bawah **n** jika nilainya terlalu kecil.

SOAL 2**Source code:**

```
def fak(n):
    hasil = 1
    for i in range(1, n+ 1):
        hasil *= i
    return hasil

def deret_cuy(n):
    for i in range(n, 0, -1):
        print(fak(i), end=" ")
        for j in range(i, 0, -1):
            print(j, end=" ")
        print( )

n = int(input("n = "))

deret_cuy(n)
```

Output:


```

PS D:\71251187_chinor\minggu07> py .\soal02.py
n = 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
○ PS D:\71251187_chinor\minggu07> █

```

Penjelasan:

Pertama, fungsi **fak(n)** digunakan untuk menghitung nilai faktorial dari suatu bilangan dengan cara mengalikan semua angka dari 1 hingga **n**. Kemudian fungsi **deret_cuy(n)** bertugas mencetak pola yang di mana perulangan utama berjalan dari **n** hingga 1 (menentukan jumlah baris), di mana setiap baris dimulai dengan hasil faktorial dari **i**, lalu dilanjutkan dengan deret angka menurun dari **i** sampai 1 menggunakan perulangan kedua. Setelah setiap baris selesai dicetak, program pindah ke baris berikutnya. Terakhir, program meminta input dari user dan memanggil fungsi **deret_cuy(n)** untuk menampilkan hasilnya.

SOAL 3

Source Code:

```

def deret_ribet(tinggi, lebar):
    angka = 1
    for i in range(tinggi):
        for j in range(lebar):
            print(angka, end = " ")
            angka += 1
        print()

tinggi = int(input("tinggi: "))
lebar = int(input("lebar: "))

deret_ribet(tinggi,lebar)

```

Output:

```
PS D:\71251187_chinor\minggu07> py .\soal03.py
tinggi: 5
lebar: 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
PS D:\71251187_chinor\minggu07> █
```

Penjelasan:

Di dalam fungsi **deret_ribet(tinggi, lebar)**, variabel **angka** diinisialisasi dengan nilai 1 sebagai awal deret, lalu dilakukan perulangan bersarang (*nested loop*) di mana loop luar **for i in range(tinggi)** mengatur jumlah baris, dan loop dalam **for j in range(lebar)** mengatur jumlah kolom pada setiap baris. Pada setiap iterasi, nilai **angka** dicetak ke layar dan kemudian ditambah 1 agar angka terus berurutan ke samping dan ke bawah. Setelah satu baris selesai dicetak, fungsi **print()** kosong digunakan untuk pindah ke baris berikutnya. Terakhir, program meminta input dari user untuk **tinggi** dan **lebar**, lalu memanggil fungsi tersebut untuk menampilkan pola angka sesuai ukuran yang diinginkan.